Agentes Inteligentes (AIN)

Práctica 1: Sesión 1 Introducción al ADK de Google

Planificación

- Vicente Julian (vjulian@upv.es)
- Carlos Carrascosa (carrasco@dsic.upv.es)

- 2 Prácticas Evaluables
  - 1ª Práctica --> entrega 21 y 22 de abril
  - 2ª Práctica --> entrega 26 y 27 de mayo

Prácticas

M / X				3-3 / 4-3	10-3 / 11-3	24-3 / 25-3	14-4 / 15-4	21-4 / 22-4	28-4 / 29-4	5-5 / 6-5	12-5 / 13-5	19-5 / 20-5	26-5 / 27-5

ADK (agent development kit)

Objetivos

- Conocer herramientas de desarrollo de agentes basados en modelos de lenguaje (LLM)
- Aprender a construir un agente de este tipo que trabaja para abordar tareas complejas.

ADK (agent development kit)

Agentes LLM

¿Qué son Agentes LLM?

- Los **agentes LLM (Large Language Model)** son sistemas inteligentes impulsados por modelos de lenguaje.
  - Realizan tareas específicas, interactuando con otros agentes y humanos
  - Aprenden del contexto para abordar problemas complejos de manera colaborativa.
  - Capacidad para interpretar instrucciones en lenguaje natural, razonar sobre ellas, y producir respuestas coherentes y útiles.

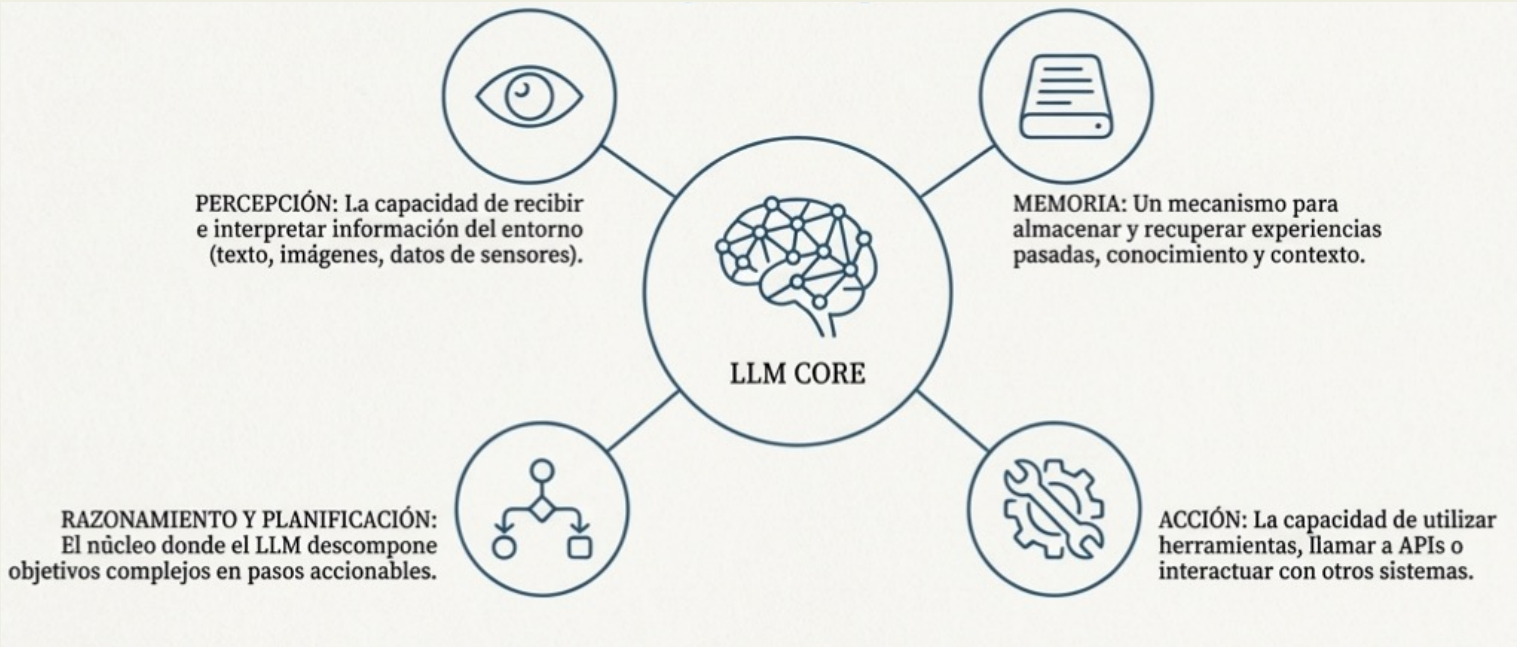
Ejemplos:

- Investigación en profundidad sobre temas específicos.
- Automatización de flujos de trabajo complejos.
- Asistentes virtuales interactivos y colaborativos.

ADK (agent development kit)

Agentes LLM

- Un agente LLM es una entidad de software que utiliza un LLM como su 'cerebro' cognitivo para percibir su entorno, tomar decisiones de forma autónoma y ejecutar acciones para alcanzar objetivos específicos.



ADK (agent development kit)

Frameworks existentes para desarrollo de Agentes LLM:

- Facilitan la creación, gestión y despliegue de equipos colaborativos de agentes.
- Permiten definir roles específicos, objetivos y flujos de interacción entre agentes.

Ejemplos: Autogen, LangGraph, ADK, CrewAI, SPADE-LLM…

ADK (agent development kit)

¿Qué es el ADK de Google?

Framework para desarrollar y desplegar agentes de IA

- Modelo-agnóstico y compatible con múltiples frameworks
- Permite coordinar agentes especializados en workflows complejos
- Evaluación integrada del rendimiento de agentes

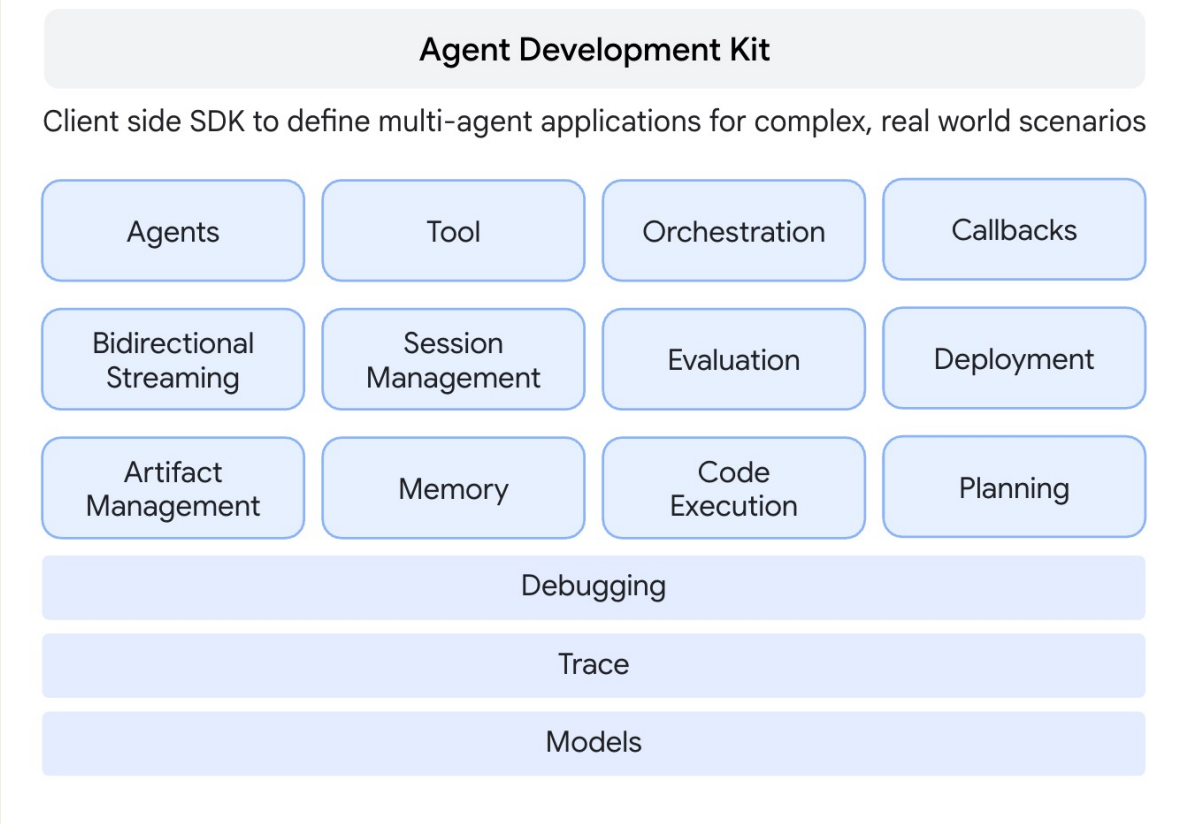
1ª versión abril de 2025

Más de 38 releases
última 1.25.1 (18/2/2026)

https://google.github.io/adk-docs/

ADK (agent development kit)

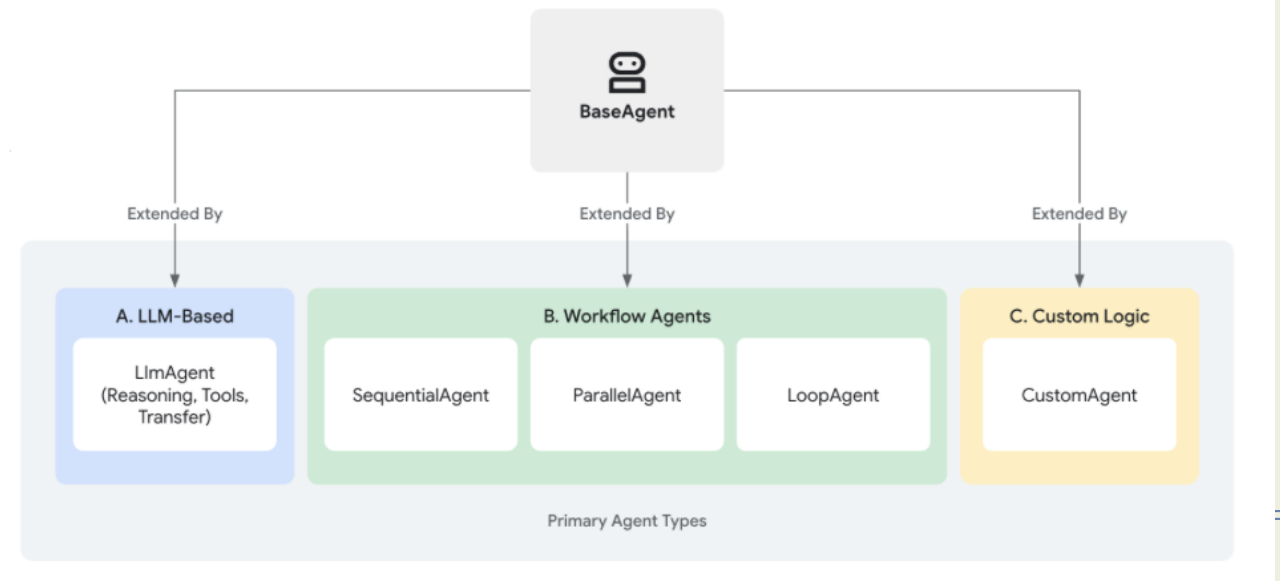
Ecosistema del ADK



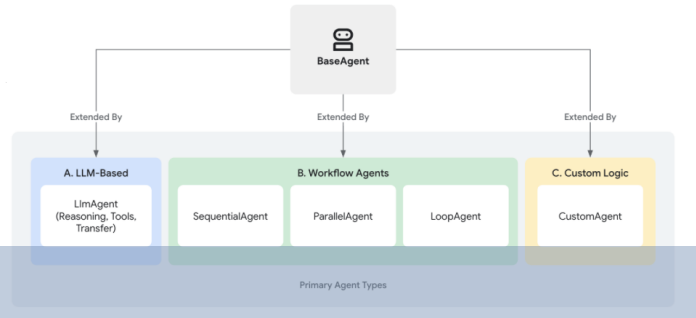
ADK Google. Agente

¿Qué es un agente en el ADK (Agent Development Kit)?

- Un agente es una unidad de ejecución autónoma que actúa para lograr uno o varios objetivos.
- Puede realizar tareas, interactuar con usuarios, usar herramientas externas y coordinarse con otros agentes.
- Todos los agentes se basan en la clase *BaseAgent*, que sirve de plantilla fundamental.



ADK Google. Agente



Tipos principales de agentes en ADK

- Agentes LLM (**LlmAgent, Agent**)
  - Motor principal: un modelo de lenguaje (LLM).
  - Función: entender lenguaje natural, razonar, planificar, usar herramientas y decidir dinámicamente qué hacer.
  - Uso típico: tareas basadas en lenguaje y decisiones flexibles (no deterministas).
- Agentes de flujo de trabajo (**SequentialAgent, ParallelAgent, LoopAgent**)
  - Motor principal: lógica predefinida (secuencia, paralelo, bucle).
  - Función: controlar la ejecución de otros agentes de forma predecible.
  - Uso típico: procesos estructurados que necesitan ejecución determinista.
- Agentes personalizados (**CustomAgent**)
  - Motor principal: código a medida.
  - Función: implementar lógicas específicas, flujos de control propios o integraciones especiales.
  - Uso típico: requisitos muy concretos que no encajan en los tipos estándar.

ADK Google. Agente

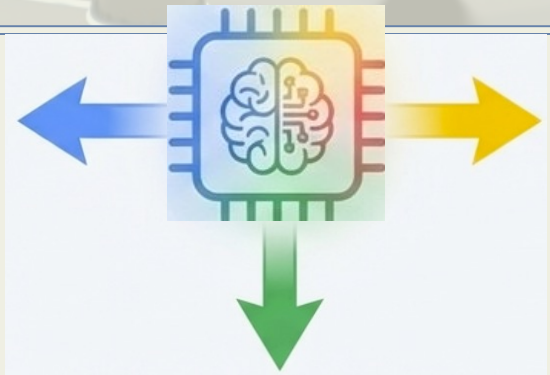
Anatomía de un Agente

- **name**: Identificador único. Este nombre puede ser importante en SMAs, donde los agentes necesitan referirse entre sí o delegarse tareas.
- **model**: El LLM que impulsa al agente (p.ej., ·gemini-2.0-flash'). ADK es agnóstico al modelo.
- **description**: Resumen de alto nivel de la especialidad del agente. Crucial para la delegación en sistemas multi-agente.
- **instructions**: El 'prompt del sistema'. Define la personalidad, el objetivo y las reglas de comportamiento del agente.
- **tools**: Una lista de las capacidades que el agente puede ejecutar (funciones, otras APls, etc.).

Ejemplo de Agente ('agent.py')

```python
from google.adk.agents import Agent

greeting_agent = Agent(
    name="greeting_agent",
    model="gemini-2.0-flash",
    description="Un agente amigable que saluda al usuario.",
    instructions=[
        "Eres un asistente servicial que saluda al usuario.",
        "Pregunta el nombre del usuario.",
        "Saluda al usuario por su nombre."
    ]
)
```

ADK Google. Herramientas



Herramientas Pre-construidas de Google

Funcionalidades listas para usar y optimizadas para el ecosistema de Google.

'google_search',
'built_in_code_execution',
'VertexAiSearchTool' (para **RAG**).

Solo **se puede usar una a la vez** y funcionan **exclusivamente con modelos Gemini.**

Herramientas de Terceros

Integración perfecta con los ecosistemas de herramientas de otros frameworks populares.

ADK proporciona clases como 'LangchainTool' y 'CrewaiTool' para una fácil importación.

Permite reutilizar herramientas existentes y aprovechar la comunidad sin reescribir código.

Herramientas Personalizadas

Funciones de Python que exponen una lógica propia, APIs o fuentes de datos.

El LLM interpreta el 'docstring' de la función para decidir cuándo y cómo usarla.

El método más común y potente para adaptar agentes a necesidades específicas.

ADK Google. Herramientas

Herramientas Pre-construidas de Google

Herramientas de Terceros

Funcior
optimiz
Google

los
ros
es.

mo
ara
ión.

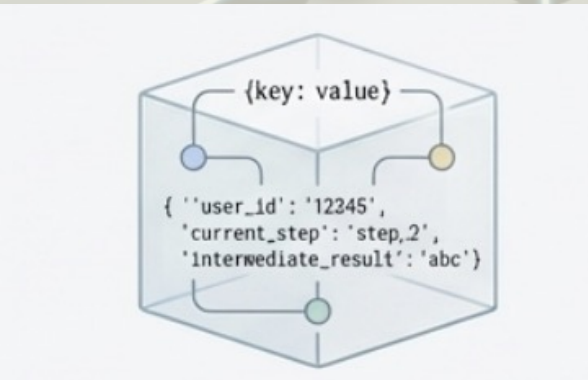
tas
dad
igo.

```python
def get_live_weather_forecast(location: str):
    """
    Obtiene el pronóstico del tiempo en vivo para una ubicación específica.

    Args:
        location: El nombre de la ciudad (p.ej., "San Francisco, CA").

    Returns:
        Un diccionario que contiene la temperatura y el pronóstico.
    """

    # ... lógica para llamar a una API de clima ...
    return {"location": location, "temperature": "22°C", "forecast": "Soleado"}
```

cuándo y cómo usarla.

El método más común y potente para adaptar agentes a necesidades específicas.

ADK Google. Memoria



Sesión (Memoria Conversacional)

¿Qué es? Un registro cronológico de todos los eventos en una interacción: mensajes del usuario, respuestas del agente, llamadas a herramientas y sus resultados.

Propósito: Proporciona el contexto inmediato de la conversación. Permite al agente entender preguntas de seguimiento.

Analogía: El historial de un chat.

Estado (Memoria Compartida)

¿Qué es? Un diccionario de Python que puede ser leído y modificado por todos los agentes y herramientas en un flujo de trabajo.

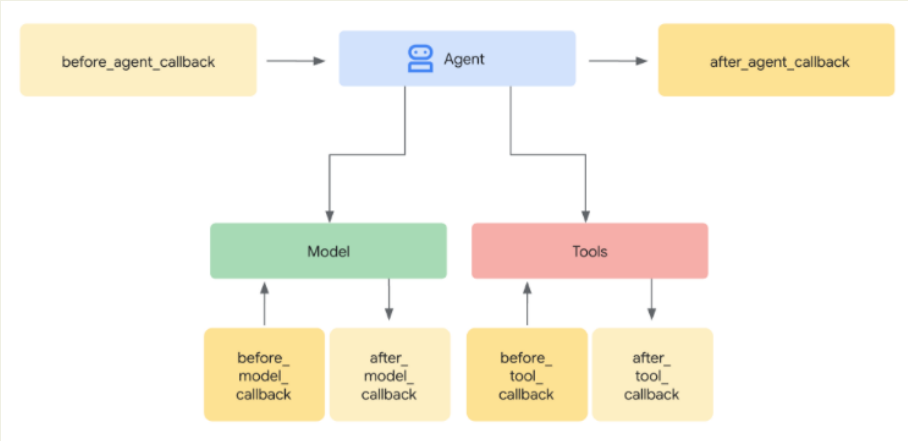
Propósito: Almacenar información clave que necesita persistir y ser compartida entre diferentes pasos (p.ej., ID de usuario, resultados intermedios).

Acceso en el Prompt: Se puede inyectar valores del estado directamente en las instrucciones usando la sintaxis '{nombre_de_la_clave}'.

ADK Google. Guardarraíles

Concepto: **Seguridad y Restricción del Comportamiento**

- Los guardarraíles son mecanismos de seguridad, control y cumplimiento
- Se implementan para restringir el comportamiento del agente y garantizar que las interacciones sean seguras, relevantes y se alineen con las políticas o reglas de negocio.
- Una forma de implementar estos controles es a través de los *Callbacks* (retrollamadas), permiten inspeccionar y modificar datos en momentos específicos del ciclo de vida del agente
- ADK ofrece diferentes tipos de *callbacks* que permiten a los desarrolladores controlar cada parte del ciclo de vida del agente




```python
from google.adk.types import LLMResponse

def filtrar_lenguaje_ofensivo(context, llm_request):
    """
    Inspecciona el último mensaje del usuario para contenido inapropiado.
    """
    # Obtener el mensaje más reciente del usuario
    ultimo_mensaje = context.get_latest_user_message().text

    if "tonto" in ultimo_mensaje.lower():
        # **Guardarraíl activado:** Retorna una respuesta LLM inmediata
        # para saltar la ejecución del modelo generativo.
        return LLMResponse(
            content="Lenguaje inapropiado. Por favor, reformula tu solicitud."
        )

    return None # Permite que el ciclo del LLM continúe [12]

# Aplicación del callback en la definición del Agente (Conceptual)
mi_agente = Agent(
    name="AgenteDeSoporte",
    model="gemini-2.5-flash",
    before_model_callback=filtrar_lenguaje_ofensivo
)
```

ADK Google. Arquitectura ejecución

Agentes, sesiones y runners

ADK Google. Arquitectura ejecución

Métodos de Ejecución del Agente en ADK

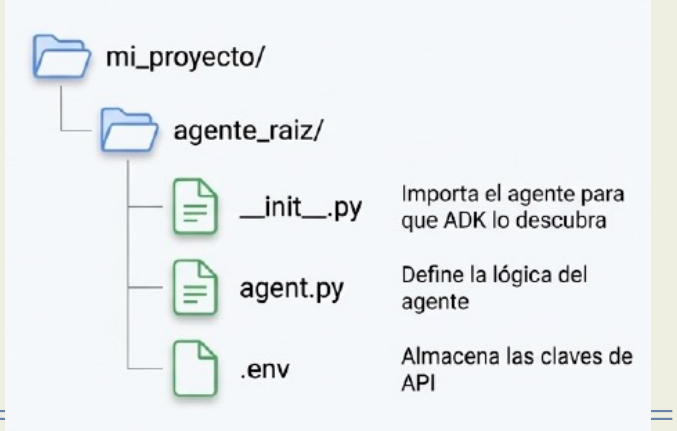
- Existen dos enfoques principales para ejecutar y probar un agente en ADK
- Adaptados a diferentes necesidades, ya sea el desarrollo rápido o la integración en producción.

1. Ejecución mediante Script de Python

- Este método requiere gestionar explícitamente los componentes del agente
- Construir y gestionar manualmente la memoria (Session) y el motor de ejecución (Runner)
- El agente se ejecuta llamando directamente a métodos del Runner, como runner.run o runner.async

2. Ejecución mediante CLI

- Este método es la forma más rápida y preferida para el desarrollo y la depuración.
- El sistema ADK se encarga automáticamente de gestionar la complejidad del backend.
- **adk web**: Inicia un servidor web local (generalmente en http://localhost:8000)
- **adk run**: Ejecuta el agente directamente en la terminal donde se puede chatear



ADK Google. Instalación

Abrir un terminal

Crear un entorno virtual de conda con Python 3.10 o superior

```
(base)$ conda create -n adk

(base)$ conda activate adk
```

Polilabs Linux: usar virtualenv

```
$ virtualenv adk

$ source adk/bin/activate
```

Instalar ADK: `(adk)$ pip install google-adk`

```
(adk)$ pip install litellm      # esto es para usar modelo LLM distinto a Gemini
```

Alternativa en Windows: Usar Anaconda para crear el entorno virtual

ADK Google. Instalación

Crear un proyecto de agente

- Ejecuta el comando *adk create* para iniciar un nuevo proyecto de agente:

```
(adk)$ adk create my_agent
```

- El proyecto de agente creado tiene una estructura, donde *agent.py* contendrá el código principal del agente.

```
$ my_agent/
    agent.py    # código principal del agente
    .env        # claves de API del proyecto
    __init__.py
```


Configurar clave de API

```
echo 'GOOGLE_API_KEY="TU_API_KEY"' > .env  # o de otro modelo
```

ADK Google. Mi primer agente

Versión CLI

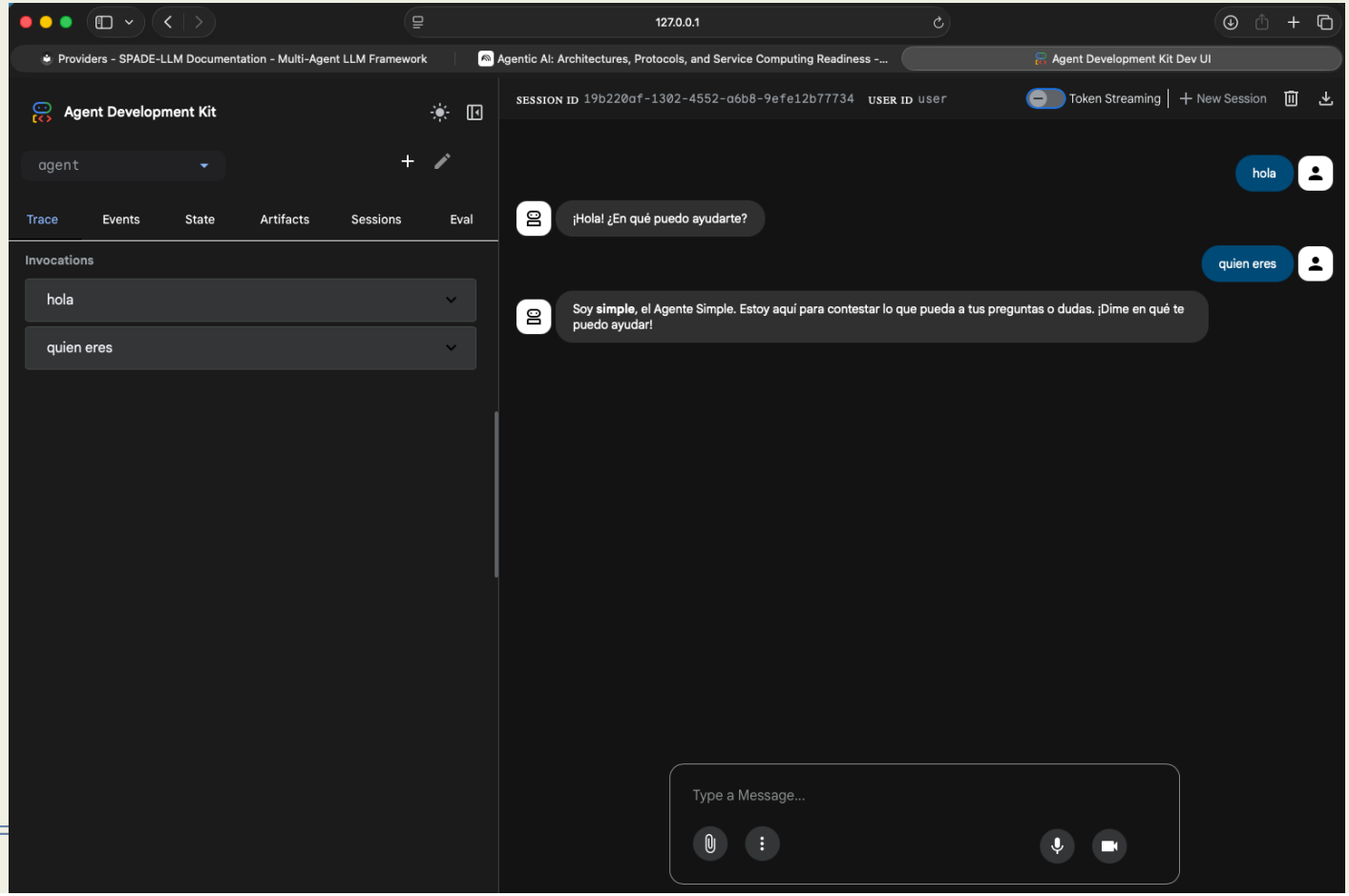
```python
from google.adk.agents import LlmAgent
from google.adk.models.lite_llm import LiteLlm

# ----------------------
# ROOT AGENT
# ----------------------

root_agent = LlmAgent(
    name="simple",
    #model="gemini-2.0-flash",
    model=LiteLlm(model="openai/gpt-oss-120b",
        api_base="https://api.poligpt.upv.es/",
        api_key="sk-LFXs1kjaSxtEDgOMlPUOpA"),
    description="Agente que contesta lo que pueda",
    instruction=(
        "Eres el Agente Simple.\n"
        "Tu tarea es contestar lo que puedas a lo que te llegue."
    ),
)
```

Modelo que vamos a usar
Requiere instalar LiteLlm
`pip install litellm`

ADK Google. Mi primer agente

```
(adk)$ adk web
```

Abrir navegador:
http://localhost:8000



ADK Google

Ejemplos

V1: ejemplo simple de un agente que únicamente contesta.

V2: ejemplo de agente que hace uso de una *tool* propia que lee noticias de un fichero Excel.

```
pip install pandas openpyxl
```

V3: ejemplo de agente que hace uso de una *tool* que busca noticias en Google news.

```
pip install feedparser
```

V4: ejemplo de agente que hace uso de un *callback* que simplemente filtra palabras malsonantes.

V5: ejemplo de agente cuestionario que usa varias *tools* propias y variables de estado.